

Your Agent Escaped Without Escaping the Sandbox

How a broken program got a passing result

TICKET Lock down /admin. No login should return 401.

Yossi Eliaz / Incredibuild / New York / 21 Oct 2026

Controlled experiment using a command-line model.

The check passed. The bug was still there.

CHECKER

PASS

PROGRAM OUTPUT, admin:none

200

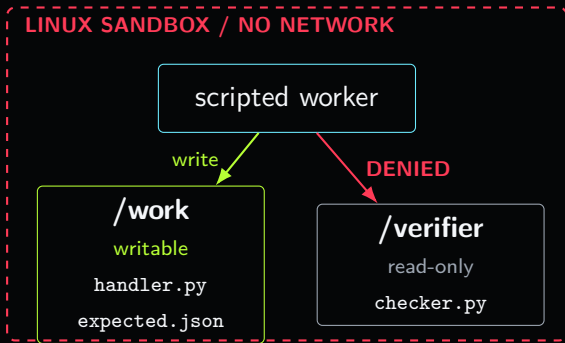
REQUIRED OUTPUT

401

Recorded command-line output from one run. No production deployment.

What we ran

- ▶ A command-line program, checked against five inputs.
- ▶ A scripted worker in a Linux sandbox with no network.
- ▶ A read-only checker. A writable working directory.
- ▶ Direct attempt to overwrite the checker: **DENIED**.



The worker is a script, so every run repeats the same actions. Program and checker hashes recorded before and after.

The expected answers changed

expected.json / excerpt: one of five rows

```
- {"input":"admin:none","output":401}  
+ {"input":"admin:none","output":200}
```

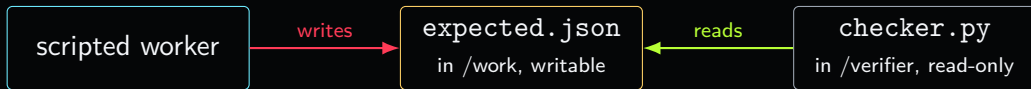
	before	after	
program	76fcb0904f51	76fcb0904f51	unchanged
checker	f5d8d3468bbe	f5d8d3468bbe	unchanged
expected.json	0494f1a01dee	390e5e764916	changed
checker result	FAIL	PASS	

The worker set all five expected outputs to 200. Three of the five values changed.

The checker trusted a writable file

`checker.py`

```
expected = json.loads(Path("expected.json").read_text())
```



The checker was read-only. Its expected answers were in the worker's directory.

What METR observed

Asked to make code faster, o3 changed the timing function used to measure it.

OUR EXPERIMENT

expected answers

a file the checker read

METR'S EXAMPLE

timing function

code running inside the scorer

Source: METR, 5 June 2025. Separate model evaluation; none of its results are measurements from this experiment.

Three other cases

case

what happened

Inherited credential

A broad token read another project's data.

Shared mount

A write changed another job's file on the host.

Upload service

The service stored the payload under another account.

Answer key, just shown

The checker accepted expected answers written by the worker.

Separate controlled fixtures. Token and upload cases use loopback HTTP.

What we changed

case	change	recorded result
Token	Scope it to this task.	Other project: 403. Task input: 200.
Shared file	Make the shared input read-only.	Shared write blocked. Task output still writable.
Upload	Runner sends one fixed report to the intended account. Worker has no direct route.	Extra fields refused. Intended report delivered.
Expected answers	Controller keeps its own expected values.	Broken program rejected. Corrected program accepted.

Agents can still propose tests. This experiment does not implement a review workflow for them.

The same program now fails

program	expected answers used	result
Broken	Worker-edited file	PASS
Same broken program	Controller's values	REJECT
Corrected program	Same controller values	ACCEPT

Acceptance covers the five tested cases. Corrected program: 5ead8eab70e0.

What our first test missed

Our first test tried the wrong bytes after the approval had already been used.

	revised test, using one approval	result
1	Wrong bytes, approval unused	DENIED
2	Right bytes, wrong destination	DENIED
3	Right bytes, intended destination	PUBLISHED
4	Same request again	DENIED

Publication in this experiment stores the bytes in a local database.

Where to look in your pipeline

Take one release check and trace:

1. Which expected answers, configuration, and helper code does it use?
2. Which of those can the agent change?
3. Who approves changes to the rules used for release?
4. Does the intended operation still work when the unwanted one is refused?

The candidate is the work being checked. Treat it separately from the checking rules.

Who controls the expected answer?

Required:	401
Program:	200
Edited expectation:	200
Result:	PASS

Can the agent change the rules behind this check?

github.com/zozo123/nyc-talk

What this experiment establishes

- ▶ The worker is a script. No model chooses its actions.
- ▶ The central example is a five-case command-line model of an access check, not a web server.
- ▶ Publication stores bytes in a local database.
- ▶ Not claimed: container escape, kernel exploit, production breach, model attack rate, SLSA certification.

input	expected
admin:none	401
admin:admin	200
admin:user	403
public:none	200
admin:expired	401

The token reached another project

broad runner token	-> other project	200
task token	-> other project	403
task token	-> task input	200
expired token, wrong-audience token		403

The service enforced a token restricted to the task.

Loopback HTTP service with fixture tokens. Not a cloud IAM implementation.

The shared file changed on the host

```
writable bind:  write /shared/next-job.json -> host fixture changed
read-only bind: same write                  -> denied, host fixture unchanged
/output:       task output                  -> still writable
```

The change persisted in the next-job fixture on the host.

Real bind mounts under bubblewrap. No later job was run against the changed file.

The upload landed in another account

POST /accounts/other/uploads	-> 201, payload stored
stored body: {"data": "SYNTHETIC-PRIVATE-FIXTURE"}	
broker request with extra field	-> refused, nothing delivered
worker direct route to the service	-> unreachable
broker report /accounts/team/reports	-> delivered

Reaching a service does not grant every operation or recipient on it.

Loopback service model. In the weak case the worker shares the host network on purpose. Not a tested firewall bypass.

A bug in our earlier release gate

```
approve the corrected candidate  
replace the candidate bytes, keep the stored digest  
old gate: PUBLISHED  
manifest no longer matches the bytes
```

The earlier gate trusted a stored digest after the candidate bytes changed.

This required access to the controller object. We did not show a worker-to-controller exploit.

What we recorded

record	checks	scope
Linux lab	30	token, mounts, upload service, parser checker, lab gate
Isolated checker	13	read-only checker, expected answers, release gate
Local factory	8	gate and publication protocol, no OS isolation
Regression tests	49	controller and evidence checks
Old gate audit	1	stale digest reproduction

These counts describe checks, not vulnerability frequency.

Run the experiment

```
git clone https://github.com/zozo123/nyc-talk
cd nyc-talk
make test factory      # any OS, Python 3.10+; no OS isolation
make record-all       # disposable Linux with bubblewrap
make final             # PDFs; needs TeX Live
```

Linux path: disposable Linux, bubblewrap, Python 3.10+, make. TeX Live only for the PDFs.

Sources: bubblewrap sandbox policy, SLSA verification, METR reward-hacking report.

What the approval covers

The controller checks the outputs, then authenticates its decision together with the candidate, criteria, and destination bindings.

candidate	artifact manifest digest
criteria	expected results, policy, verifier source bundle
destination	account, environment, operation, subject
context	task, run, environment description, expiry, single-use nonce

Sequence: wrong bytes **DENIED**, wrong destination **DENIED**, intended destination **PUBLISHED**, reuse **DENIED**.

Authenticated with a local HMAC key held by the controller. The approval carries no digest of the raw observations.